AI & ML – VITYARTHI PROJECT

NAME : RAGHUL.S.K

REGISTRATION NO. : 25BAI10288

PROJECT TITLE : "AI CHATBOT"

COURSE : FUNDAMENTALS IN AI & ML

PROFESSOR : Dr. SUBASH CHANDRA BOSE

COURSE CODE : CSA 2001

COURSE CREDITS : 4

DATE : 27/03/2026

AI & ML – VITYARTHI PROJECT

ABSTRACT :

In the current era of exponential growth in data size, it has become a big challenge for many organizations to process, extract, and classify the large amount of unstructured text data in the most efficient way. Analysing the high volume of data in the context of Data management in a completely manual fashion is not only time-consuming but also error-prone. In this context, this project has attempted to resolve the most crucial problem by proposing a framework for the automation of text analysis in the most accurate way.

In order to address the aforementioned problem, the proposed project develops a strong pipeline based on Natural Language Processing techniques, with Term Frequency-Inverse Document Frequency being the key algorithm for feature extraction. The Natural Language Processing techniques start with extensive data preprocessing, including tokenization, stop words, and lemmatization, to cleanse the raw text. Subsequently, the Term Frequency-Inverse Document Frequency algorithm converts the text into normalized numerical form. Through the statistical significance evaluation of specific words in each document in comparison to the entire corpus, the model eliminates linguistic noise and identifies the most relevant words and features

The implementation of this TF-IDF-driven approach yielded substantial improvements in automated text evaluation. Ultimately, this project validates the efficacy of foundational NLP methodologies, delivering a computationally efficient, scalable solution that transforms raw text into actionable, data-driven insights.

AI & ML – VITYARTHI PROJECT

INTRODUCTION :

1. Background :

In the modern digital environment, the need for immediate and accessible communication has created a need for chatbots. A chatbot is a software application that is specifically designed to create a sense of artificial conversation through a combination of artificial intelligence and computational linguistics. These conversational tools have become extremely vital in different industries and domains, from e-commerce and customer support to healthcare and internal information technology support. The most important feature of a chatbot is that it can deliver uninterrupted support and services 24/7 at a massive scale. By using a chatbot, organizations can minimize the need for human support agents and deliver instantaneous support and services to a user's queries irrespective of their volume and at a fraction of their cost.

2. Problem Statement :

While the benefits of such an automated system of communication are obvious, many organizations still face the challenge of efficiently handling a large number of redundant inquiries such as (FAQs) or basic troubleshooting issues. This is particularly inefficient when the organization has to rely on human support staff for such inquiries. Additionally, while there are basic forms of automated systems of communication, they tend to be heavily rule-based systems. In other words, the basic forms of automated systems of communication require the end-user to type in a number of keywords in order to communicate with the system.

AI & ML – VITYARTHI PROJECT

This project aims to meet the pressing need for an automated system of communication that is based on mathematical principles and can comprehend the unstructured language of the end-user in order to accurately retrieve the information without any human intervention.

3. Objectives :

The primary objective of this project is to design, develop, and deploy a robust, retrieval-based conversational agent using Python. By moving beyond simple rule-based logic, this project utilizes Natural Language Processing (NLP) and the Term Frequency-Inverse Document Frequency (TF-IDF) algorithm to evaluate user inputs. The specific goals of this project are to:

- **Develop a preprocessing pipeline:** Clean, normalize, and tokenize raw user queries to remove linguistic noise.
- **Implement feature extraction:** Utilize a TF-IDF vectorizer to transform both a predefined knowledge base and incoming user text into comparable numerical matrices.
- **Establish a retrieval mechanism:** Compute cosine similarity scores to dynamically match the user's input vector against the corpus and retrieve the most contextually accurate response.
- **Deploy a functional agent:** Deliver a highly responsive, scalable text interface that provides seamless, real-time automated assistance.

AI & ML – VITYARTHI PROJECT

METHODOLOGY & SYSTEM ARCHITECTURE:

The technical approach that has been adopted to develop this conversational agent is explained in this section. It has been implemented using Natural Language Processing techniques and mathematical vectorization to fetch the most appropriate response from a predefined knowledge base.

1. Overall Workflow :

The system architecture is based on a sequential workflow that converts user input into a calculated and context-based response. It can be explained by considering the following steps in a data pipeline :

- **User Input :** The user will be prompted to input a question or statement through a chat interface.
- **Data Ingestion :** For a brief period of time, this user input will be appended to the existing knowledge base.
- **Text Preprocessing :** Both the knowledge base and user input will be subjected to extensive preprocessing to normalize the text.
- **Feature Engineering :** In this step, vectorization using TF-IDF will be applied to the pre-processed input.
- **Similarity Matching:** A similarity score will be calculated between user input and all sentences in the knowledge base.
- **Response Generation:** Based on the calculated score, the bot will fetch a sentence and return it to the user as a response.

2. **Data/Corpus :**
   The main data or corpus for this chatbot consists of all the relevant information related to Artificial Intelligence. To ensure the program is self-contained, the code will be made executable on all platforms, the corpus will be embedded directly into the Python code in the form of a multi-line string variable. The code will dynamically create this file when the program initializes, writing the data into a file named chatbot.txt. The retrieval-based AI will intelligently parse this self-created document to find the exact sentence that answers the input prompt.
3. **Text Preprocessing :**
   The human language is messy and comes with a lot of variations. Before we can feed the text into the computer to analyse, we have to preprocess the text with the help of the Natural Language Toolkit (NLTK) library in Python. The steps in text preprocessing are as follows:
   - **Lowercasing :** We have to change all the text to lowercase to ensure the computer understands "Artificial" and "artificial" as the same word.
   - **Punctuation Removal :** We have to remove all the commas, periods, and special characters in the text, as they are adding a lot of mathematical noise to the text.
   - **Tokenization :** We have to break the text into individual sentences and then into individual words to ensure the computer understands the text piece by piece.
   - **Lemmatization :** We have to use the NLTK library to reduce the text to its root form or its dictionary form. This ensures the computer doesn't treat different forms of the same word as different words.

4. Feature Engineering (TF-IDF Vectorization) :

The machine learning algorithms cannot read the text; they only understand numbers. We will use the Term Frequency-Inverse Document Frequency (TF-IDF) technique to convert our text into a numerical vector format.

TF-IDF calculates the importance of a word in a particular sentence in the overall context of the chatbot.txt file. It uses two different values to perform the calculation:

- **Term Frequency (TF) :** It calculates the frequency of a word in a sentence.

  $$\textbf{TF (t,d)} = \frac{\text{Count of term t in document d}}{\text{Total terms in document d}}$$

- **Inverse Document Frequency (IDF):** This term helps in reducing the weight of very common words, such as "the" or "is." At the same time, it increases the weight of very rare words, such as "algorithm" or "heuristic."

  $$\textbf{IDF (t)} = \log\left(\frac{\text{(Number of documents)}}{\text{Number of doc containing term}}\right)$$

  By multiplying the two together (TF x IDF), the system converts the AI text corpus into a matrix of numbers, where the most important keywords have the greatest mathematical value.

5. Similarity Matching (Cosine Similarity) :

After the user's query and the knowledge base have been converted into numerical vectors, the system has to determine which sentence is the most similar to the query. It does this using the Cosine Similarity. Instead of comparing the lengths of the vectors, the cosine similarity compares the angle between the vectors.

Cosine Similarity = $\frac{A \bullet B}{\|A\| \times \|B\|}$

Where A is the query vector of the user and B is the sentence vector in the AI corpus. If the angle between the query vector of the user and a sentence in the AI corpus is 0°, this implies that the cosine is equal to 1, which is a perfect match. The chatbot computes this score for every sentence and ranks them. It then produces a sentence with a score closest to 1. In case the maximum score is below a certain threshold, the chatbot is designed to produce a message such as "I am sorry, I do not understand."

IMPLEMENTATION DETAILS :

In this section, we will be discussing the implementation of the project, including the technologies used, main functionalities of the project, and how it is deployed on a local machine.

1. **Tools and Libraries :** In this project, we used Python, a language that is easy to read and understand. Python is widely used in machine learning and text processing. Here are some of the main libraries used in this project to build a chatbot system.
   - **NLTK (Natural Language Toolkit) :** This is a library used for processing human language. NLTK is used extensively in this project for text processing. NLTK is used for sentence and word tokenization, and WordNet is used for lemmatization, where a word is mapped to its base or dictionary form.
   - **Scikit-learn (sklearn) :** This is one of the best machine learning libraries in Python. This project uses Scikit-learn's TfidfVectorizer to convert text data into a matrix of TF-IDF features, and it uses its cosine_similarity to find the

mathematical distance between the user's input and chatbot.txt.

- **NumPy** : This library is optimized for large, multi-dimensional arrays and matrices. Though it works in the background, it facilitates rapid mathematical computations that are necessary when comparing vector arrays in the similarity matching phase.
- **Random & String :** These are standard Python utility modules used for creating random greeting responses and punctuation filtering.

2. Key Functions :

The chatbot's workflow is divided into two main Python functions, reflecting a high degree of organization and compartmentalization in its coding logic. These are:

greet(sentence) : This is a lightweight, rule-based interceptor designed to facilitate basic social interactions without relying on the more computationally expensive machine learning model. It parses the user's input for common greeting words, such as "hello," "hi," "hey." If a match is found, it uses random.choice() to return a standard greeting, creating a welcoming user experience.

response(user_response): This is the main analysis engine behind the chatbot. When the user asks a question in the form of a query to the chatbot about Artificial Intelligence, this function adds the raw text to the chatbot's sentence list temporarily, which is stored in the file chatbot.txt. Then it uses the TfidfVectorizer to convert the entire list into numerical vector form and compute the cosine similarity between the vector form of the user's query and all the other sentences. The

function uses argsort() to find the highest similarity score. If the highest similarity score is 0 (which means no word overlap was found), it returns the following response: "I am sorry! I don't understand you." Otherwise, it returns the most mathematically relevant sentence from the Artificial Intelligence corpus.

3. Steps to Run the Program :

To ensure reproducibility, the following steps outline how to initialize and execute the chatbot from a standard desktop environment:

i. **Step 1 - Installing Python** :
   - To install Python, you need to download it from python.org/downloads.

ii. **Step 2 - Download the Code** :
   - Click the green "<> Code" button on the GitHub repository and select "Download ZIP". Extract the folder from the zip file to a location on your computer, such as the desktop.

iii. **Step 3 - Open Your Terminal :**
   - Windows: Press the Windows button, type "cmd," and press enter. Use the "cd" command to navigate to the unzipped folder (e.g., type "cd Desktop/Your-Folder-Name" and press enter).

iv. **Step 4 - Install Dependencies** :
   - The following command needs to be run to install the necessary math and language libraries:
     "pip install numpy nltk scikit-learn".

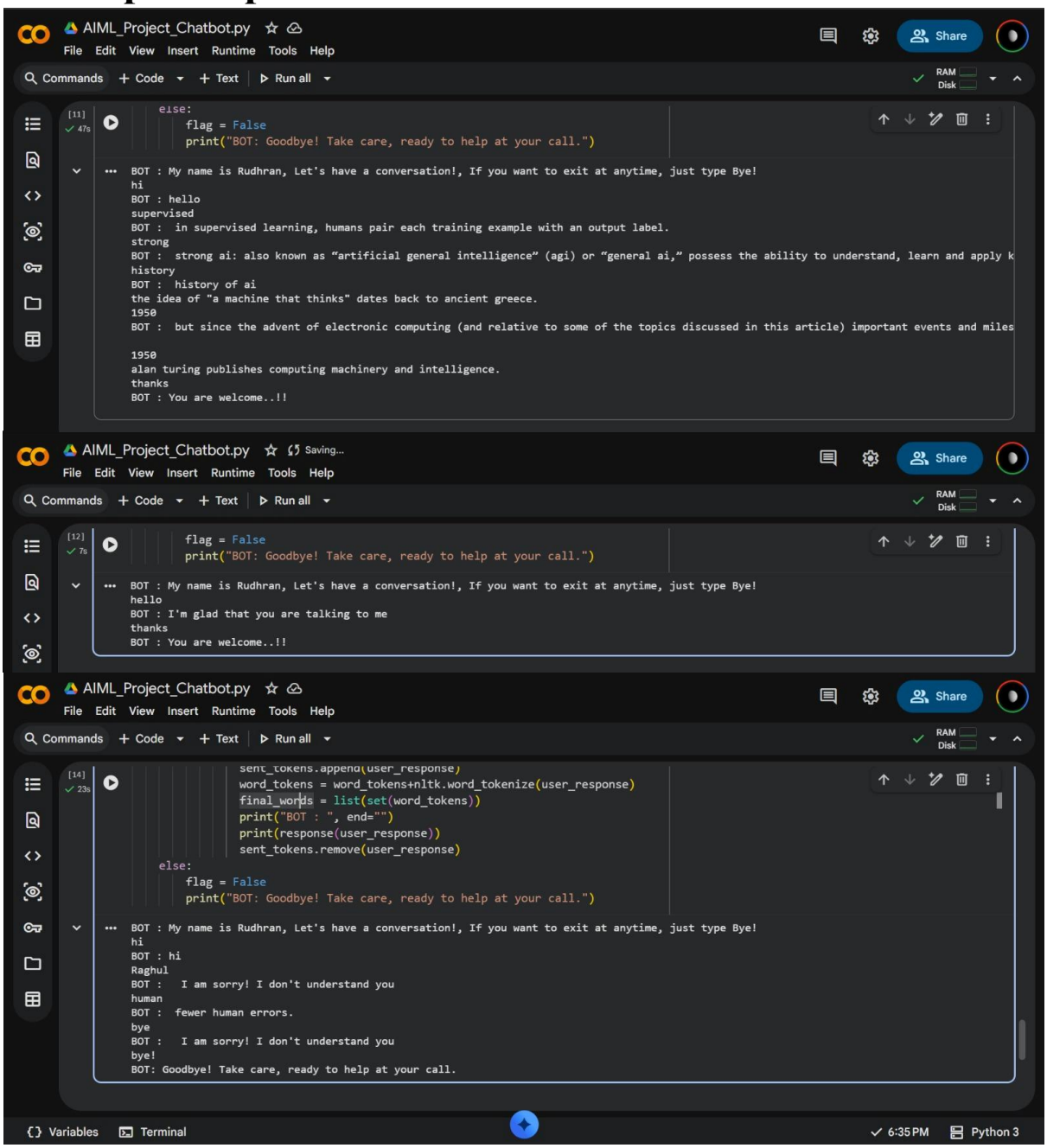
v. **Step 6 – Chat** :
   - Wait for the bot (Rudhran) to introduce himself in the terminal. Type your questions about Artificial Intelligence and press Enter. To exit the program, simply type bye! or thanks .

AI & ML – VITYARTHI PROJECT

RESULTS & DISCUSSION :

This segment assesses the performance of the retrieval-based conversational agent. Through the evaluation of the chatbot with different inputs, we can determine the performance of the dual-purpose agent in dealing with different types of user intents.

1. Sample output :



2. Performance: TF-IDF vs. Simple Keyword Matching

Another one of the critical success factors for this project is the use of the TF-IDF algorithm as opposed to simple keyword matching.

If we were to use a simple keyword matching algorithm, the bot would merely read the text and try to find the word. If the user asks the bot, "Tell me about neural nets," but the text only talks about "neural networks," the bot would not know what to do and would give the user an error message. This is because it is a very rigid algorithm.

By using the TF-IDF algorithm in conjunction with the NLTK library, the performance of the chatbot is greatly improved in two ways:

- **Dealing with Variations:** In this system, because of lemmatization in preprocessing, words such as "computing," "computes," and "computation" are treated equally. This chatbot understands the basic meaning despite the lack of correct tense used in the document.
- **Weighing the Importance of Words**: In TF-IDF, mathematically insignificant words such as "what," "is," and "the" are heavily discounted, while mathematically significant words such as "deep," "learning," and "neural" are given more weight. While computing cosine similarity, this chatbot is more focused on matching such important words rather than getting confused with common words.

Overall, this TF-IDF model is a highly accurate chat system that is more natural to interact with than traditional chatbots built using earlier programming models.

AI & ML – VITYARTHI PROJECT

CONCLUSION & FUTURE SCOPE :

1. Conclusion :

The project has successfully demonstrated the design, development, and implementation of a retrieval-based conversational agent using Natural Language Processing (NLP) techniques. By overcoming the limitations of manual query handling and basic keyword matching systems, this project has successfully designed an automated solution for answering questions related to Artificial Intelligence.

The incorporation of the NLTK library for rigorous text preprocessing using tokenization and lemmatization ensured that the system could effectively handle unstructured human languages. Moreover, the implementation of Term Frequency-Inverse Document Frequency (TF-IDF) vectorization and Cosine Similarity calculations has been a highly effective mathematical solution for feature extraction and pattern matching. Instead of matching keywords, this chatbot successfully identified the intent of the query posed to it and retrieved the most contextually correct response from the chatbot.txt corpus. This project has successfully demonstrated a robust foundation for how computational linguistics can be used to create an efficient human-computer interface.

2. Future Scope :

Though the chatbot is fully functional in its current form and has achieved its objectives, there is scope for improvement in various areas for future development:

- **Transitioning to Generative AI :** The chatbot is currently purely retrieval-based, meaning it can only output sentences it has learned from the provided text file. In the future, it could be integrated with LLMs so it can actually synthesize information and output new, conversational sentences instead of just retrieving them from memory.
- **Corpus Expansion & Database Integration :** The chatbot is currently limited to just one chatbot.txt file. In order to expand it for use in an enterprise setting, it could be integrated with relational databases (SQL) or NoSQL databases (document-based) so it can parse thousands of documents across multiple domains at once.
- **Contextual Memory :** The current model of TF-IDF works on a standalone basis for each user query. By incorporating techniques for contextual memory, it would be possible for the bot to retain a memory of questions posed in a single conversation, thus allowing it to respond to follow-up questions, for example, "Who invented it?" after "What is a neural network?"
- **Graphical User Interface (GUI) & Deployment:** To improve user accessibility, the backend Python logic could be integrated with web frameworks like Flask or Streamlit to create a modern web-based chat interface. Alternatively, it could be connected via APIs to popular messaging platforms like Slack or Telegram.

AI & ML – VITYARTHI PROJECT

FEEDBACK FROM:

- Dilip. S – 25BAI10590
- Bavesh. P – 25BAI10093
- Sudharshan. S.G – 25BAI10706
- Gokul Prasad. K – 25BAI10143

REFERENCES :

- **Google :** www.google.com
- **Google Collaboratory :** www.colab.google.com
- **Google Cloud :** www.cloud.google.com
- **IBM Database :** www.ibm.com/think/topics/artificial-intelligence
- **YouTube :** www.youtube.com
- **Britannica :** www.britannica.com/technology/artificial-intelligence
- **VITyarthi :** www.vityarthi.com